



A Type-Safe Arithmetic Expression Interpreter in Haskell

A project assignment report submitted to the

Department of Electrical and Information Engineering
Faculty of Engineering
University of Ruhuna
Sri Lanka

EC 8206 Functional Programming

by

Achintha H.G.R. - EG/2021/4384

October 2026

1 Introduction and How to Run

This report describes a small, type-safe arithmetic expression interpreter in Haskell [1], compares it with an imperative version and reflects on the functional programming principles behind it. Every module compiles without warnings under GHC 9.6.7 with `-Wall`. Only `cabal test` needs more than GHC (it downloads QuickCheck).

```
git clone https://github.com/Raviyaa27/functional-programming-project
cd "functional-programming-project/Project Files/haskell"
runghc Main.hs      # demo: samples, simplify, batch report
runghc Tests.hs     # 72 expected-vs-actual unit tests
cabal test          # the unit tests and 8 QuickCheck properties
python ../imperative/imperative_eval.py    # Python 3.10+
```

2 Design and Implementation

2.1 Language Design

```
data Expr = Lit Double | Var String
          | Add Expr Expr | Sub Expr Expr | Mul Expr Expr | Div Expr Expr
          | Let String Expr Expr
          | If BExpr Expr Expr
data BExpr = BLit Bool | Less Expr Expr | Equal Expr Expr
          | And BExpr BExpr | Or BExpr BExpr | Not BExpr
type Env   = [(String, Double)]
```

Listing 1: The expression types (abridged from `Expr.hs`, lines 27-47).

`Expr` is a sum type with eight alternatives, each carrying exactly the fields it needs: besides the six required constructors, `Let` adds a local binding and `If` a choice. Conditions have their own type, `BExpr`, so numbers and conditions cannot be mixed. In an `Env`, `lookup` finds the first match, so new bindings shadow old ones.

2.2 Evaluation

```
51 eval :: Env -> Expr -> Either String Double
52 eval _ (Lit n)      = Right n
53 eval env (Var x)    = maybe (Left ("undefined variable: " ++ x)) Right
54                      (lookup x env)
55 eval env (Add a b)   = (+) <$> eval env a <*> eval env b
56 eval env (Sub a b)  = (-) <$> eval env a <*> eval env b
57 eval env (Mul a b)  = (*) <$> eval env a <*> eval env b
58 eval env (Div a b)  = do
59   x <- eval env a
60   y <- eval env b
61   if y == 0
62     then Left ("division by zero: " ++ pretty (Div a b))
63     else Right (x / y)
64 eval env (Let x e body) = do
65   v <- eval env e
66   eval ((x, v) : env) body
67 eval env (If c t e)   = do
68   b <- evalB env c
69   if b then eval env t else eval env e
```

Listing 2: The evaluator (`Expr.hs`, lines 51-69).

Table 1: Sample evaluations: expected vs. actual output of `Main.hs`.

#	Expression	Expected	Actual	OK
1	<code>(x + 3) * y</code>	16	16	✓
2	<code>x / y</code>	2.5	2.5	✓
3	<code>let a = x * 2 in a + 1</code>	11	11	✓
4	<code>let x = 1 in (let x = 10 in x) + x</code>	11	11	✓
5	<code>if x < 10 && not (y == 0) then x / y else 0</code>	2.5	2.5	✓
6	<code>if z == 0 then 0 else x / z</code>	0	0	✓
7	<code>x / (y - 2)</code>	error: division by zero: x / (y - 2)	error: division by zero: x / (y - 2)	✓
8	<code>x + w</code>	error: undefined variable: w	error: undefined variable: w	✓
9	<code>let a = 1 / z in a + 1</code>	error: division by zero: 1 / z	error: division by zero: 1 / z	✓

`eval` has one equation per constructor and recurses on the sub-expressions. Errors are values of type `Either String Double`: the arithmetic cases use applicative style and `Div`, `Let` and `If` use do-notation in the `Either` monad [2], so the first `Left` stops evaluation and becomes the result. An undefined variable (line 53) and division by zero (lines 58-63) are reported this way, never as exceptions. The mutually recursive `evalB` (lines 71-81) evaluates conditions with short-circuiting `And/Or`, and `If` evaluates only the chosen branch.

2.3 Simplification and Higher-Order Functions

`simplify = rewrite . descend` (lines 86-119) simplifies the children first and then applies the first matching rule at the root: nine identities ($x + 0$, $0 + x$, $x - 0$, $x - x$, $x \cdot 1$, $1 \cdot x$, $x \cdot 0$, $0 \cdot x$, $x/1$), constant folding (never of $n/0$, which `eval` must report), if with a constant condition, and removal of an unused `let`; `simplifyB` adds double negation and constant comparisons. Being bottom-up, one pass reaches a normal form, e.g. $x * 1 + 0 \rightarrow x$.

```
evalAll :: Env -> [Expr] -> [Either String Double]
evalAll env = map (eval env)
evalBatch env = rights . evalAll env
batchReport env = foldr tally (BatchReport 0 0 [] []) . evalAll env
where
  tally (Right v) r = r { succeeded = succeeded r + 1
                        , values     = v : values r }
  tally (Left err) r = r { failed    = failed r + 1
                        , failures   = err : failures r }
```

Listing 3: The list functions (excerpt of `Expr.hs`, lines 144-164).

`eval env` is a partially applied (curried) function `Expr -> Either String Double` that `map` applies to each expression (line 145). `evalBatch` composes it with `rights`, `batchReport` counts successes and failures in one `foldr` (6 and 3 on the samples), and `undefinedVars` uses `filter` with the section (``notElem` bound`) (line 188) to find unbound variables statically.

3 Sample Evaluations and Testing

Table 1 compares hand-computed results for the nine samples ($x = 5$, $y = 2$, $z = 0$) with the output of `Main.hs`. `Tests.hs` adds 72 unit tests and `Props.hs` eight QuickCheck properties [3] on 1000 random expressions each; all pass.

Table 2: The Haskell and Python versions compared.

	Haskell (<i>Expr.hs</i>)	Python (<i>imperative_eval.py</i>)
Error representation	<code>Left msg</code> , a value; the type shows that failure is possible	<code>EvalError</code> exception; not visible in the signature
Error propagation	<code><*></code> and <code>do</code> stop at the first <code>Left</code> ; callers must unwrap the result	Stack unwinding to whichever <code>try</code> encloses the call
Mutable state	None: <code>let</code> builds a new list (<code>x, v</code>) : <code>env</code>	<code>result</code> reassigned; <code>env</code> overwritten, restored in <code>finally</code>
Misspelt or missing case	Compile error (<code>Mull</code> not in scope); <code>-Wincomplete-patterns</code> warning	Found only if that branch runs (<code>case _</code>)
Mixing numbers and conditions	Type error: <code>If</code> needs a <code>BExpr</code> , <code>Add</code> needs <code>Exprs</code>	Accepted: any value is “truthy”, and <code>True + 3</code> gives 4

4 Comparative Analysis

Listing 4 is the imperative version: a `match` on string tags, a mutable `result`, a shared dictionary and exceptions.

Errors. In Haskell, failure is part of the type of `eval`, so a caller must handle the `Left` before using the `Double`, and `<*>` and `do` propagate it at no cost. In Python, failure is invisible in the signature and a forgotten `except` crashes the program. **State.** Python reassigns `result` in every case and overwrites the shared `env` in `let`; without the `finally`, an error in a `let` body would leave the binding behind. Haskell has no mutable state. **Bugs ruled out.** `imperative_eval.py` runs five programs that GHC rejects before they run (the last rows of Table 2, plus a string literal that crashes with `TypeError`). Division by zero and unbound variables depend on run-time values, so neither version excludes them by types; Haskell returns them as values instead of throwing them.

```
def evaluate(env, expr):
    result = None
    match expr[0]:
        case "lit":
            result = expr[1]
        case "var":
            if expr[1] not in env:
                raise EvalError("undefined variable: " + expr[1])
            result = env[expr[1]]
        case "add":
            result = evaluate(env, expr[1]) + evaluate(env, expr[2])
        case "sub":
            result = evaluate(env, expr[1]) - evaluate(env, expr[2])
        case "mul":
            result = evaluate(env, expr[1]) * evaluate(env, expr[2])
        case "div":
            left = evaluate(env, expr[1])
            right = evaluate(env, expr[2])
            if right == 0:
                raise EvalError("division by zero")
            result = left / right
        case "let":
            name = expr[1]
            had_old, old = name in env, env.get(name)
            env[name] = evaluate(env, expr[2])
            try:
                result = evaluate(env, expr[3])
            finally:
                if had_old:
                    env[name] = old
                else:
                    del env[name]
        case _:
            raise EvalError("unknown expression tag: " + str(expr[0]))
    return result
```

Listing 4: Imperative evaluator (`imperative_eval.py`, lines 10-57; the condition cases are omitted).

5 Reflection

Pure functions. A pure function’s result depends only on its arguments, and it has no side effects. The type of `eval` (line 51) has no `IO`, so it cannot print, read input or change any variable; the same environment and expression always give the same result, and likewise for `simplify`. This made testing simple: a unit test just compares values, and QuickCheck can compare `eval env e` with `eval env (simplify e)` thousands of times. Functions are also values: `eval env` is passed to `map` (line 145) [4].

Immutability. Nothing is updated in place. Line 66, `eval ((x, v) : env)` body, does not modify `env`: it builds a new list whose tail *is* the old one (shared, not copied). When the `let` ends, the caller still holds its original `env`, so the binding disappears with no clean-up (sample 4) and Python’s `try/finally` has no counterpart. `simplify` likewise builds a new tree (lines 92-97) and leaves its input intact, so `Main.hs` prints each expression before and after simplification from the same value.

Why invalid expressions cannot be constructed. Each constructor fixes the number and types of its fields: `Add` takes exactly two `Exprs`, and `If` takes a `BExpr` and two `Exprs`. An operator with a missing operand, or a misspelt one such as `Mull`, is therefore not an `Expr` at all, and keeping conditions in a separate type turns Python’s silent bugs into compile errors (`compile_errors.txt`):

```
bad1 = If (Lit 1) (Lit 2) (Lit 3)
      * Couldn't match expected type `BExpr' with actual type `Expr'
```

`Add (Less (Lit 1) (Lit 2)) (Lit 3)` fails likewise with the types swapped, and a function that forgets a constructor gets a warning naming it (`Patterns of type `Expr' not matched: If _ _ _`). Errors that depend on values (an unbound name, a zero divisor) are beyond the types; `eval` returns them as `Left`, and `undefinedVars` finds unbound names before evaluation (QuickCheck confirms it is sound and complete). `TypedExpr.hs` goes further with a GADT indexed by the result type, so a single expression type still rejects ill-typed terms at compile time.

What testing revealed. QuickCheck refuted the claim that `simplify` never changes whether evaluation succeeds by finding `w - w`: it fails with `undefined variable: w` but simplifies to 0, because rules that discard a sub-expression (`x · 0`, `x - x`, an unused `let`) discard its error too. What holds, on 1000 random expressions, is that every *successful* result is preserved; because failure is a value, this was visible and testable.

References

- [1] G. Hutton, *Programming in Haskell*. Cambridge, UK: Cambridge University Press, 2nd ed., 2016.
- [2] P. Wadler, “Monads for functional programming,” in *Advanced Functional Programming*, vol. 925 of *Lecture Notes in Computer Science*, pp. 24-52, Springer, 1995.
- [3] K. Claessen and J. Hughes, “QuickCheck: A lightweight tool for random testing of Haskell programs,” in *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP)*, pp. 268-279, 2000.
- [4] J. Hughes, “Why functional programming matters,” *The Computer Journal*, vol. 32, no. 2, pp. 98-107, 1989.